Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado pede funções de sincronização em C, com Pthreads, para o embarque de passageiros em vagões de metrô, em que cada vagão e cada passageiro é uma thread. O sync_car.c com station_init, station_load_car, station_wait_for_car e station_on_board, usando um único mutex e duas variáveis de condição. O programa foi testado com o sync_car-runner.c fornecido.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
station_init, station_load_car, station_wait_for_car e station_on_board em sync_car.c	OK	make; ./sync_car → Looks good!
Usar só pthread_mutex_init/lock/unlock e pthread_cond_init/wait/signal/broadcast	OK	grep -n pthread_ sync_car.c
Não usar semáforos nem outras primitivas	OK	grep -n sem_ sync_car.c (sem resultado)
Um único lock por struct station	OK	sync_car.h: um só pthread_mutex_t
Nunca mais de um carro na estação por vez	OK	Assumido: a struct guarda um só vagão
Embarque simultâneo: vários passageiros saem de station_wait_for_car antes do primeiro station_on_board	OK	pthread_cond_broadcast no station_load_car; ./sync_car com vagões de até 49 assentos
Sem busy-wait	OK	Toda espera é pthread_cond_wait dentro de while
station_load_car retorna na hora sem passageiros ou sem assentos	OK	./sync_car: chamadas iniciais do runner com 0 e 10 assentos sem passageiros, e com 0 assentos e 100 passageiros
Compilação por Makefile gerando sync_car	OK	make clean; make
README com arquivos, compilação, execução, testes e sistema operacional	OK	README.md no repositório e no .tar
Relatório em PDF	OK	svv.pdf

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

```
for i in $(seq 1 20); do ./sync_car || break; done
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.c → implementação de station_init, station_load_car, station_wait_for_car e station_on_board.**
2. **sync_car.h → struct station (quant_assentos_livres, quant_passageiros, quant_embarcando, o mutex e as duas variáveis de condição) e os protótipos.**
3. **sync_car-runner.c → programa de teste, sem alterações.**
4. **Makefile → compila sync_car.c e sync_car-runner.c com -pthread, gera sync_car e tem a regra clean.**
5. **README.md → arquivos, como compilar, executar e testar, e sistema operacional.**
6. **evidencias.log → registro do terminal gravado com script.**

Decisões técnicas:

1. **Decisão: duas variáveis de condição, cond_passageiro e cond_vagao, com o mesmo mutex → Motivo: são dois tipos de espera, o passageiro esperando vagão com assento e o vagão esperando poder partir → Consequência: cada aviso chega só a quem interessa, e um signal para o vagão não acorda um passageiro.**
2. **Decisão: pthread_cond_broadcast para acordar os passageiros quando o vagão chega → Motivo: o vagão não consegue escolher quantos acordar, e pthread_cond_signal acordava só um → Consequência: todos acordam e testam o while de novo; enquanto há assento, embarcam, e quem não tem lugar volta a dormir.**
3. **Decisão: contador quant_embarcando, separado de quant_assentos_livres → Motivo: quant_assentos_livres mostra quem ganhou assento, mas não quem já sentou, e o vagão partia antes dos station_on_board → Consequência: station_on_board diminui o contador e acorda o vagão quando ele chega a 0, e o while do vagão espera também enquanto há gente embarcando.**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Estudo antes de codar	Estudei o capítulo 6 do Operating System Concepts (seção crítica, mutex, semáforos, monitores com variáveis de condição, liveness) e li o runner do professor antes de escrever código	-
S2	Implementar a primeira ideia	Modelei a struct com vagao, quant_assentos_livres e quant_passageiros.	-
S3	Contagem de embarque	Adicionei quant_embarcando mais tarde na implementaca: aumenta quando o passageiro ganha assento em station_wait_for_car e diminui em station_on_board, que acorda o vagão quando chega a 0	O runner deu returned early!: o vagão partia quando os assentos acabavam, antes de os passageiros sentarem
S4	Remoção de redundâncias	Revisei o código depois dos testes passarem e tirei três partes sem efeito: quant_passageiros == 0 no while do passageiro, o pthread_cond_signal no fim do station_wait_for_car e o campo vagao, substituído por quant_assentos_livres	Com quant_embarcando, essas partes ficaram redundantes; o signal acordava o vagão cedo e ele voltava a dormir

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S2	Modelagem com três contadores: vagao, quant_assentos_livres e quant_passageiros	Parcial	Faltava contar quem ganhou assento e ainda não chamou station_on_board; previsão: returned early!	03/11/2026, antes das 10:38	Sem registro no log (decisão no papel); a struct aparece no commit 9fbe1bc "feat: sync_car e struct station", às 10:38
2	S2	Escrever sync_car.h e station_init	Falha	Header salvo como sync_car-runner.h, tipo boolen, falta de ; e protótipos sem void; no station_init, & nas atribuições e NULL faltando no pthread_cond_init	03/11/2026, 10:38 a 11:18	Commits e5c9114 "fix:esqueci o tipo de retorno" (10:58) e c6cbb94 "feat: mutex e condicoes na struct + fix de header" (11:07); no evidencias.log, sessão das 11:20, cat sync_car.h e gcc -Wall -c sync_car.c sem erros
3	S2	Laço de espera do station_wait_for_car	Falha	O while terminava em ;, ficando com corpo vazio: espera ocupada. Corrigi colocando pthread_cond_wait dentro do laço	03/11/2026, até 12:04	O ; foi visto em revisão, sem registro no log; o commit 0d96822 "feat:in inicio de station_wait_for_car com o laço de espera" (12:04) já tem a correção

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
4	S2	Primeira compilação com as quatro funções	Falha	Erros de compilação: : no lugar de ; em quant_assentos_livres--, statio-> no lugar de station->, e campos sem station-> no if do station_wait_for_car. Depois, undefined reference to station_load_car: o nome da função estava escrito statio_load_car	03/11/2026, sessão do log das 16:12	evidencias.log, sessão das 16:12: erros de compilação e undefined reference to station_load_car
5	S2	Rodar o runner	Falha	Teste inicial travou: station_load_car não retornava sem passageiros. Faltava o retorno imediato; no lugar dele havia um while com pthread_cond_wait. Numa tentativa intermediária, escrevi = no lugar de == na condição (erro lvalue required)	03/11/2026, entre 16:12 e 16:37	evidencias.log: "did not return immediately when no waiting passengers"; commit 60c6294 (16:37)
6	S2	Rodar o runner depois do retorno imediato	Falha	Vagão de 46 assentos: passengers not to board. pthread_cond_signal acordava só um passageiro; troquei por pthread_cond_broadcast. A primeira execução depois da troca repetiu o erro porque eu não tinha recompilado	03/11/2026, entre 16:37 e 16:57	evidencias.log: "passengers not to board" duas vezes; commit c6d635f (16:57)
7	S2	Rodar o runner com broadcast	Falha	station_load_car failed nos vagões de 1 e 34 assentos: no while do vagão, neguei cada parte da regra de partida mas mantive o ||. Corrigi com De Morgan, trocando por &&	03/11/2026, entre 16:37 e 16:57	evidencias.log: "station_load_car failed" duas vezes; commit c6d635f "fix: acorda todos os passageiros com broadcast e corrige

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
						condição de partida do vagão" (16:57)
8	S2	Rodar o runner com o while corrigido	Falha	returned early! no vagão de 32 assentos: o vagão partia quando os assentos acabavam, antes dos station_on_board. Era a lacuna prevista na linha 1	03/11/2026, antes das 16:57	evidencias.log: "station_load_car returned early!"
9	S3	quant_embarcando: ++ em station_wait_for_car, -- em station_on_board com signal ao chegar a 0, e while do vagão com || quant_embarcando != 0. Uma versão intermediária com && quant_embarcando foi corrigida antes de rodar	Sucesso	-	03/11/2026, entre 16:57 e 17:14	evidencias.log; commit 5837b41 "fix: corrige o embarque e a partida do vagão" (17:20)
10	S3	20 execuções seguidas do runner	Sucesso	-	03/11/2026, antes das 17:14	evidencias.log: 20 execuções com "Looks good!"
11	S2	Makefile	Sucesso	-	03/11/2026, sessão do log das 17:19; commit às 17:24	evidencias.log: criação do Makefile com printf. O teste com make clean, make e ./sync_car não ficou no log, só em print. Commit b330446 "feat: adiciona Makefile" (17:24)
12	S4	Remover quant_passageiros == 0 do while do	Sucesso	-	03/11/2026, sessão do log das 19:36 às	evidencias.log, sessão das 19:36: 2 execuções

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
		passageiro, o pthread_cond_signal do fim do station_wait_for_car e o campo vagao			19:43; commit às 19:41	com "Looks good!" depois da refatoração; commit 4f12e22 "refactor: remove o campo vagao e condições redundantes" (19:41)

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

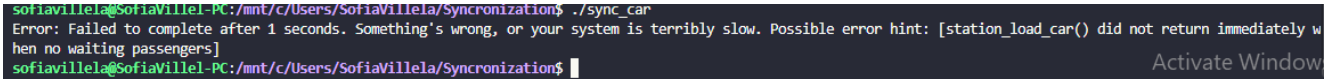
No início da sessão de trabalho, rode:

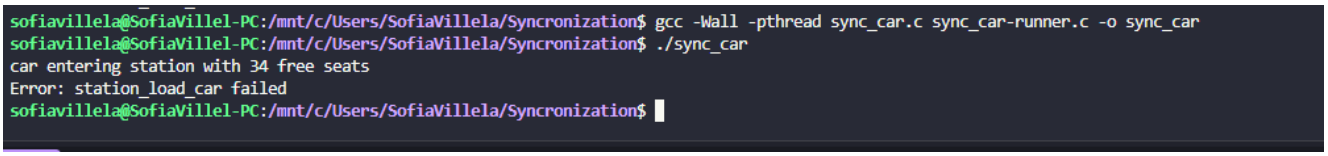
```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

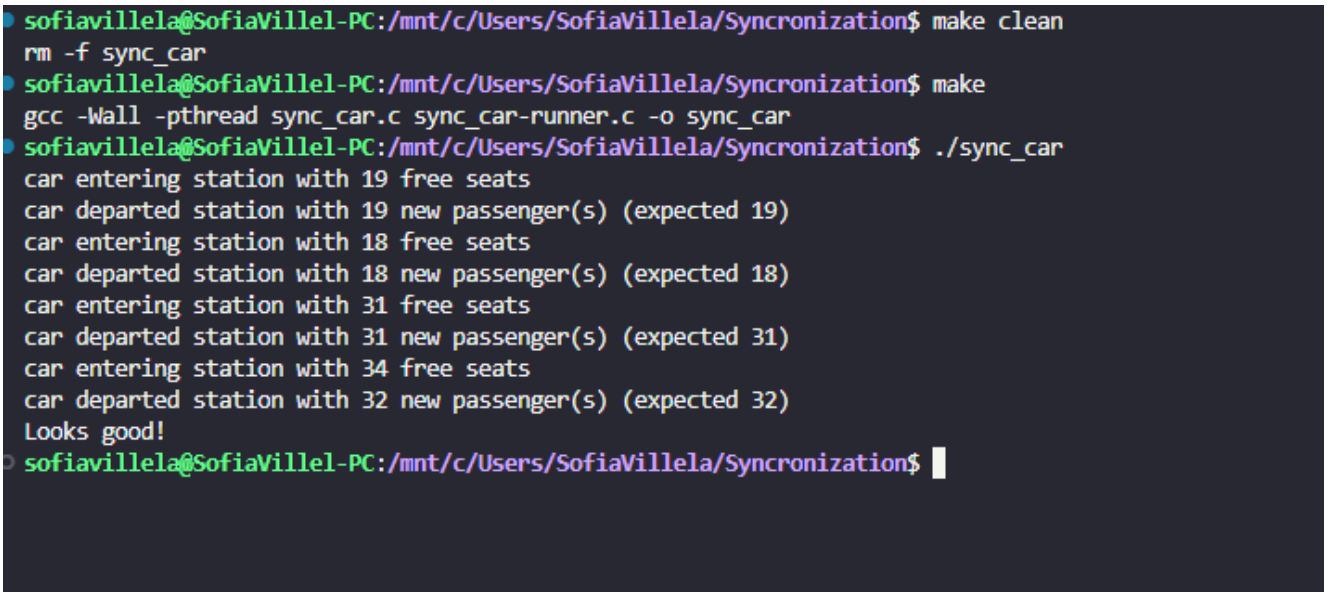
6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.





- Validação final: teste passando / saída correta.



Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Usei o Claude para estudar o capítulo 6 do Operating System Concepts seção por seção e para entender o sync_car-runner.c. Na implementação, pedi revisão de Pthreads, sem que ele me dissesse os erros de lógica; esses eu investiguei com perguntas guiadas..

Prompts principais (liste, sem história):

quero que me ensine esse capitulo completo (enviei o capitulo 6 do livro)

transforma em pseudocodigo o teste runner para eu entender o que ele ta fazendo

meus testes estao dando errado nao consigo identificar pq ve (mandei o print do terminal do erro e o claude me fez perguntas guiadas para eu descobrir e arrumar o codigo)

reune nesse script todos os horarios e arruma com base nessa tabela(enviei a tabela as estrategias do diario de tentativa ainda sem o Quando? pois nao sabia exatamente os horarios ja que fiz em um dia)

O que validei manualmente e como:

Rodei o runner depois de cada execucao e anotei cada mensagem de erro. Para achar as causas, simulei no papel os valores dos contadores em cada while: no teste 1, no vagão de 46 assentos e na condição do vagão, entre outras. No fim rodei 20 execuções seguidas com for i in $(seq 1 20); do ./sync_car || break; done, todas terminando com Looks good!. Testei o Makefile com make clean, make e ./sync_car.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A parte mais difícil foi transformar a regra de partida do vagão em código: errei a negação da condição do while e só entendi depois de simular os valores. Entendi por que o pthread_cond_wait fica dentro de um while e por que o vagão precisa de broadcast. Na próxima, vou escrever as condições de espera no papel antes de codar e recompilar sempre antes de testar.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim (make clean, make)
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim: vagão com 0 assentos e vagão sem passageiros, nos testes do runner
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Rodaria 500 execuções seguidas do runner, que comecei mas não registrei o resultado. Escreveria testes próprios além do runner, como passageiros chegando enquanto o vagão carrega e vários vagões seguidos com poucos assentos. Testaria com mais de 100 passageiros e com vagões maiores que os 49 assentos do runner. Mediria o custo do pthread_cond_broadcast, que acorda todos os passageiros a cada vagão, mesmo os que vão voltar a dormir, quando há muitos passageiros esperando.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.